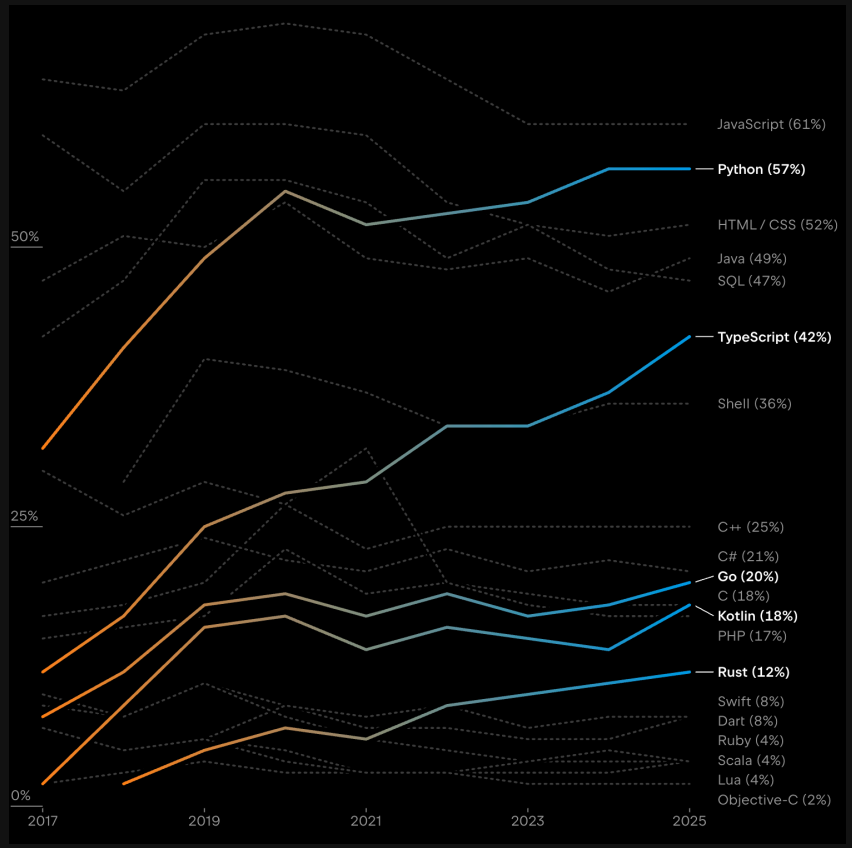
TypeScript utenfor komfortsonen

Er TypeScript relevant i 2025?

- 99,208,449 Weekly Downloads (npmjs.com)
- Mest brukte språk på Github i 2025
- JetBrains Tools & Trends 2025: "Among the most dramatic rises in real-world usage over the last five years is TypeScript."



Vi skal gå gjennom små og store eksempler, og jeg håper at alle kan ta med seg minst en ny teknikk tilbake til oppdrag 🚀

Template literals i TypeScript

```
// JavaScript Template Literals
const color = "red";
const size = "medium";
const className = `${color}-${size}`; // "red-medium"

// Template literal types i TypeScript
type Color = "red" | "green" | "blue";
type Size = "small" | "medium" | "large";
type ColoredSize = `${Color}-${Size}`;
// Result: 'red-small' | 'red-medium' | 'red-large' | ...

// Eksempel 1: Event Handler Types
type Action = "click" | "hover" | "focus" | "blur" | "change";
type EventHandler = `on${Capitalize<Action>}`;
// Result: "onClick" | "onHover" | "onFocus" | "onBlur" | "onChange"

function handleEvent(event: EventHandler) {
  // Type-safe event handling
}
handleEvent("onClick"); // ✅
handleEvent("click"); // 🚫
```

Template literals i TypeScript

```
// JavaScript Template Literals
const color = "red";
const size = "medium";
const className = `${color}-${size}`; // "red-medium"

// Template literal types i TypeScript
type Color = "red" | "green" | "blue";
type Size = "small" | "medium" | "large";
type ColoredSize = `${Color}-${Size}`;
// Result: 'red-small' | 'red-medium' | 'red-large' | ...

// Eksempel 1: Event Handler Types
type Action = "click" | "hover" | "focus" | "blur" | "change";
type EventHandler = `on${Capitalize<Action>}`;
// Result: "onClick" | "onHover" | "onFocus" | "onBlur" | "onChange"

function handleEvent(event: EventHandler) {
  // Type-safe event handling
}
handleEvent("onClick"); // ✅
handleEvent("click"); // 🚫
```

Template literals i TypeScript

```
4   const className = `${color}-${size}`; // "red-medium"
5
6   // Template literal types i TypeScript
7   type Color = "red" | "green" | "blue";
8   type Size = "small" | "medium" | "large";
9   type ColoredSize = `${Color}-${Size}`;
10  // Result: 'red-small' | 'red-medium' | 'red-large' | ...
11
12  // Eksempel 1: Event Handler Types
13  type Action = "click" | "hover" | "focus" | "blur" | "change";
14  type EventHandler = `on${Capitalize<Action>}`;
15  // Result: "onClick" | "onHover" | "onFocus" | "onBlur" | "onChange"
16
17  function handleEvent(event: EventHandler) {
18    // Type-safe event handling
19  }
20  handleEvent("onClick"); // ✅
21  handleEvent("click"); // 🚫
22
23  // Eksempel 2: CSS Class Variants
24  type Variant = "primary" | "secondary";
```

Template literals i TypeScript

```typescript
type Color = "red" | "green" | "blue";
type Size = "small" | "medium" | "large";
type ColoredSize = `${Color}-${Size}`;
// Result: 'red-small' | 'red-medium' | 'red-large' | ...

// Eksempel 1: Event Handler Types
type Action = "click" | "hover" | "focus" | "blur" | "change";
type EventHandler = `on${Capitalize<Action>}`;
// Result: "onClick" | "onHover" | "onFocus" | "onBlur" | "onChange"

function handleEvent(event: EventHandler) {
  // Type-safe event handling
}
handleEvent("onClick"); // ✅
handleEvent("click"); // 🚫

// Eksempel 2: CSS Class Variants
type Variant = "primary" | "secondary";
type Size = "sm" | "md" | "lg";
type ButtonClass = `btn-${Variant}-${Size}`;
// Result: "btn-primary-sm" | "btn-primary-md" | "btn-secondary-lg" | ...
```

Template literals i TypeScript

```
type Color = "red" | "green" | "blue";
type Size = "small" | "medium" | "large";
type ColoredSize = `${Color}-${Size}`;
// Result: 'red-small' | 'red-medium' | 'red-large' | ...

// Eksempel 1: Event Handler Types
type Action = "click" | "hover" | "focus" | "blur" | "change";
type EventHandler = `on${Capitalize<Action>}`;
// Result: "onClick" | "onHover" | "onFocus" | "onBlur" | "onChange"

function handleEvent(event: EventHandler) {
  // Type-safe event handling
}
handleEvent("onClick"); // ✅
handleEvent("click"); // 🚫

// Eksempel 2: CSS Class Variants
type Variant = "primary" | "secondary";
type Size = "sm" | "md" | "lg";
type ButtonClass = `btn-${Variant}-${Size}`;
// Result: "btn-primary-sm" | "btn-primary-md" | "btn-secondary-lg" | ...
```

Branded Types

```ts
const getJournal = async (patientId: string, journalId: string) =>
  await fetch(`api/patient/${patientId}/journal/${journalId}`);

const patientId = "12345";
const journalId = "67890";

await getJournal(journalId, patientId); // Feil rekkefølge!

// Løsning: Branded Types
declare const __brand: unique symbol;
type Brand<T, B> = T & { [__brand]: B };

type PatientId = Brand<string, "PatientId">;
type JournalId = Brand<string, "JournalId">;

const getJournal = async (patientId: PatientId, journalId: JournalId) =>
  await fetch(`api/patient/${patientId}/journal/${journalId}`);

const patientId = "12345" as PatientId;
const journalId = "67890" as JournalId;
```

Branded Types

```ts
const getJournal = async (patientId: string, journalId: string) =>
  await fetch(`api/patient/${patientId}/journal/${journalId}`);

const patientId = "12345";
const journalId = "67890";

await getJournal(journalId, patientId); // Feil rekkefølge!

// Løsning: Branded Types
declare const __brand: unique symbol;
type Brand<T, B> = T & { [__brand]: B };

type PatientId = Brand<string, "PatientId">;
type JournalId = Brand<string, "JournalId">;

const getJournal = async (patientId: PatientId, journalId: JournalId) =>
  await fetch(`api/patient/${patientId}/journal/${journalId}`);

const patientId = "12345" as PatientId;
const journalId = "67890" as JournalId;
```

Branded Types

```ts
const getJournal = async (patientId: string, journalId: string) =>
  await fetch(`api/patient/${patientId}/journal/${journalId}`);

const patientId = "12345";
const journalId = "67890";

await getJournal(journalId, patientId); // Feil rekkefølge!

// Løsning: Branded Types
declare const __brand: unique symbol;
type Brand<T, B> = T & { [__brand]: B };

type PatientId = Brand<string, "PatientId">;
type JournalId = Brand<string, "JournalId">;

const getJournal = async (patientId: PatientId, journalId: JournalId) =>
  await fetch(`api/patient/${patientId}/journal/${journalId}`);

const patientId = "12345" as PatientId;
const journalId = "67890" as JournalId;

```

Branded Types

```
  await fetch(`api/patient/${patientId}/journal/${journalId}`);

const patientId = "12345";
const journalId = "67890";

await getJournal(journalId, patientId); // Feil rekkefølge!

// Løsning: Branded Types
declare const __brand: unique symbol;
type Brand<T, B> = T & { [__brand]: B };

type PatientId = Brand<string, "PatientId">;
type JournalId = Brand<string, "JournalId">;

const getJournal = async (patientId: PatientId, journalId: JournalId) =>
  await fetch(`api/patient/${patientId}/journal/${journalId}`);

const patientId = "12345" as PatientId;
const journalId = "67890" as JournalId;

// Type error: Argument of type 'JournalId' is not assignable to parameter of type 'PatientId'
```

Branded Types

```
3
4   const patientId = "12345";
5   const journalId = "67890";
6
7   await getJournal(journalId, patientId); // Feil rekkefølge!
8
9   // Løsning: Branded Types
10  declare const __brand: unique symbol;
11  type Brand<T, B> = T & { [__brand]: B };
12
13  type PatientId = Brand<string, "PatientId">;
14  type JournalId = Brand<string, "JournalId">;
15
16  const getJournal = async (patientId: PatientId, journalId: JournalId) ⇒
17    await fetch(`api/patient/${patientId}/journal/${journalId}`);
18
19  const patientId = "12345" as PatientId;
20  const journalId = "67890" as JournalId;
21
22  // Type error: Argument of type 'JournalId' is not assignable to parameter of type 'PatientId'
23  await getJournal(journalId, patientId); // 🚫 Type error!
```

Branded Types

```
const patientId = "12345";
const journalId = "67890";

await getJournal(journalId, patientId); // Feil rekkefølge!

// Løsning: Branded Types
declare const __brand: unique symbol;
type Brand<T, B> = T & { [__brand]: B };

type PatientId = Brand<string, "PatientId">;
type JournalId = Brand<string, "JournalId">;

const getJournal = async (patientId: PatientId, journalId: JournalId) ⇒
  await fetch(`api/patient/${patientId}/journal/${journalId}`);

const patientId = "12345" as PatientId;
const journalId = "67890" as JournalId;

// Type error: Argument of type 'JournalId' is not assignable to parameter of type 'PatientId'
await getJournal(journalId, patientId); // 🚫 Type error!
```

Branded Types

```
const patientId = "12345";
const journalId = "67890";

await getJournal(journalId, patientId); // Feil rekkefølge!

// Løsning: Branded Types
declare const __brand: unique symbol;
type Brand<T, B> = T & { [__brand]: B };

type PatientId = Brand<string, "PatientId">;
type JournalId = Brand<string, "JournalId">;

const getJournal = async (patientId: PatientId, journalId: JournalId) ⇒
  await fetch(`api/patient/${patientId}/journal/${journalId}`);

const patientId = "12345" as PatientId;
const journalId = "67890" as JournalId;

// Type error: Argument of type 'JournalId' is not assignable to parameter of type 'PatientId'
await getJournal(journalId, patientId); // 🚫 Type error!
```


Record<string, User>;
Record<UserId, User>;

Typeof & Keyof

```ts
const minKonto = {
  saldo: 100,
  eier: "Ola Normann",
};

type KontoInferred = typeof minKonto;

/*
type KontoInferred = {
  saldo: number;
  eier: string;
}
*/

interface KontoSkjema {
  id: string;
  eier: {
    fornavn: string;
    etternavn: string;
  };
  saldo: number;
  transaksjoner: unknown[];
```

Typeof & Keyof

```ts
const minKonto = {
  saldo: 100,
  eier: "Ola Normann",
};

type KontoInferred = typeof minKonto;

/*
type KontoInferred = {
  saldo: number;
  eier: string;
}
*/

interface KontoSkjema {
  id: string;
  eier: {
    fornavn: string;
    etternavn: string;
  };
  saldo: number;
  transaksjoner: unknown[];
```

Typeof & Keyof

```
/*
type KontoInferred = {
  saldo: number;
  eier: string;
}
*/

interface KontoSkjema {
  id: string;
  eier: {
    fornavn: string;
    etternavn: string;
  };
  saldo: number;
  transaksjoner: unknown[];
}

type SkjemaFelter = keyof KontoSkjema;

/*
type SkjemaFelter = "id" | "eier" | "saldo" | "transaksjoner"
*/
```

Typeof & Keyof

```
}
*/

interface KontoSkjema {
  id: string;
  eier: {
    fornavn: string;
    etternavn: string;
  };
  saldo: number;
  transaksjoner: unknown[];
}

type SkjemaFelter = keyof KontoSkjema;

/*
type SkjemaFelter = "id" | "eier" | "saldo" | "transaksjoner"
*/

export const DebounceConstants = {
  createNewRow: {
    none: 0,
    veryShort: 200
```

Typeof & Keyof

```ts
type SkjemaFelter = "id" | "eier" | "saldo" | "transaksjoner"
*/

export const DebounceConstants = {
  createNewRow: {
    none: 0,
    veryShort: 200,
    short: 1000,
    medium: 5000,
    long: 15000,
  },
  cellEdited: {
    short: 300,
    medium: 750,
    long: 1200,
    maxWait: 30000,
  },
} as const;

export type CreateNewRowDebounceTime =
  keyof typeof DebounceConstants.createNewRow;

/*
```

Typeof & Keyof

```ts
export const DebounceConstants = {
  createNewRow: {
    none: 0,
    veryShort: 200,
    short: 1000,
    medium: 5000,
    long: 15000,
  },
  cellEdited: {
    short: 300,
    medium: 750,
    long: 1200,
    maxWait: 30000,
  },
} as const;

export type CreateNewRowDebounceTime =
  keyof typeof DebounceConstants.createNewRow;

/*
type CreateNewRowDebounceTime = "none" | "veryShort" | "short" | "medium" | "long"
*/
```

Typeof & Keyof

```
    cellEdited: {
      short: 300,
      medium: 750,
      long: 1200,
      maxWait: 30000,
    },
  } as const;

  export type CreateNewRowDebounceTime =
    keyof typeof DebounceConstants.createNewRow;

  /*
  type CreateNewRowDebounceTime = "none" | "veryShort" | "short" | "medium" | "long"
  */

  function debounceNewRow(delay: CreateNewRowDebounceTime) {
    const timeout = DebounceConstants.createNewRow[delay];
    return timeout;
  }

  debounceNewRow("short"); // ✅
  debounceNewRow("nonexistent"); // 🚫 Type error!
```

Bruk Extends for å stille krav til generics

```tsx
interface ListProps<T extends { id: number }> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
}

function List<T extends { id: number }>({ items, renderItem }: ListProps<T>) {
  return (
    <>
      {items.map(item => (
        <div key={item.id}>  {/* TypeScript knows 'id' exists */}
          {renderItem(item)}
        </div>
      ))}
    </>
  );
}

// ✅ Works
<List items={[{ id: 1, name: "Alice" }]} renderItem={item => item.name} />

// ❌ Compile error - missing 'id'
<List items={[{ name: "Alice" }]} renderItem={item => item.name} />
```

Bruk Extends for å stille krav til generics

```tsx
interface ListProps<T extends { id: number }> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
}

function List<T extends { id: number }>({ items, renderItem }: ListProps<T>) {
  return (
    <>
      {items.map(item => (
        <div key={item.id}>  {/* TypeScript knows 'id' exists */}
          {renderItem(item)}
        </div>
      ))}
    </>
  );
}

// ✅ Works
<List items={[{ id: 1, name: "Alice" }]} renderItem={item => item.name} />

// ❌ Compile error - missing 'id'
<List items={[{ name: "Alice" }]} renderItem={item => item.name} />
```

Bruk Extends for å stille krav til generics

```tsx
interface ListProps<T extends { id: number }> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
}

function List<T extends { id: number }>({ items, renderItem }: ListProps<T>) {
  return (
    <>
      {items.map(item => (
        <div key={item.id}>  {/* TypeScript knows 'id' exists */}
          {renderItem(item)}
        </div>
      ))}
    </>
  );
}

// ✅ Works
<List items={[{ id: 1, name: "Alice" }]} renderItem={item => item.name} />

// ❌ Compile error - missing 'id'
<List items={[{ name: "Alice" }]} renderItem={item => item.name} />
```

Const assertion

```typescript
const list = [1, 2, 3];
// TypeScript inferer typen av list til number[]

list.push(4);
/*
Siden lista kan endres så er typen fleksibel til å tillate flere tall.
Dette er en liste med tall, og TS vet ikke hvor mange eller hvilke verdier.
*/

const list2 = [1, 2, 3] as const;
/*
TypeScript inferer typen av list2 til readonly [1, 2, 3]

Dette er en liste med tall, og TS vet at det kun er disse tre tallene.
*/

list2.push(4); // 🚫 Error: Property 'push' does not exist on type 'readonly [1, 2, 3]'.

// Ved å bruke 'as const' så vet vi at lista er endelig, og TypeScript kan derfor gi en mer spesifikk type.

// Eksempel
// Uten 'as const'
```

Const assertion

```ts
const list = [1, 2, 3];
// TypeScript inferer typen av list til number[]

list.push(4);
/*
Siden lista kan endres så er typen fleksibel til å tillate flere tall.
Dette er en liste med tall, og TS vet ikke hvor mange eller hvilke verdier.
*/

const list2 = [1, 2, 3] as const;
/*
TypeScript inferer typen av list2 til readonly [1, 2, 3]

Dette er en liste med tall, og TS vet at det kun er disse tre tallene.
*/

list2.push(4); // 🚫 Error: Property 'push' does not exist on type 'readonly [1, 2, 3]'.

// Ved å bruke 'as const' så vet vi at lista er endelig, og TypeScript kan derfor gi en mer spesifikk type.

// Eksempel
// Uten 'as const'
```

Const assertion

```
// TypeScript inferer typen av list til number[]

list.push(4);
/*
Siden lista kan endres så er typen fleksibel til å tillate flere tall.
Dette er en liste med tall, og TS vet ikke hvor mange eller hvilke verdier.
*/

const list2 = [1, 2, 3] as const;
/*
TypeScript inferer typen av list2 til readonly [1, 2, 3]

Dette er en liste med tall, og TS vet at det kun er disse tre tallene.
*/

list2.push(4); // 🚫 Error: Property 'push' does not exist on type 'readonly [1, 2, 3]'.

// Ved å bruke 'as const' så vet vi at lista er endelig, og TypeScript kan derfor gi en mer spesifikk type.

// Eksempel
// Uten 'as const'
const MENU_ITEMS = [
  { label: "Hjem", path: "/" },
```

Const assertion

```
list.push(4);
/*
Siden lista kan endres så er typen fleksibel til å tillate flere tall.
Dette er en liste med tall, og TS vet ikke hvor mange eller hvilke verdier.
*/

const list2 = [1, 2, 3] as const;
/*
TypeScript inferer typen av list2 til readonly [1, 2, 3]

Dette er en liste med tall, og TS vet at det kun er disse tre tallene.
*/

list2.push(4); // 🚫 Error: Property 'push' does not exist on type 'readonly [1, 2, 3]'.

// Ved å bruke 'as const' så vet vi at lista er endelig, og TypeScript kan derfor gi en mer spesifikk type.

// Eksempel
// Uten 'as const'
const MENU_ITEMS = [
  { label: "Hjem", path: "/" },
  { label: "Profil", path: "/profil" },
  { label: "Innstillinger", path: "/innstillinger" }
```

Const assertion

```
list2.push(4); // ❌ Error: Property 'push' does not exist on type 'readonly [1, 2, 3]'.

// Ved å bruke 'as const' så vet vi at lista er endelig, og TypeScript kan derfor gi en mer spesifikk type.

// Eksempel
// Uten 'as const'
const MENU_ITEMS = [
  { label: "Hjem", path: "/" },
  { label: "Profil", path: "/profil" },
  { label: "Innstillinger", path: "/innstillinger" },
];

type MenuItem = (typeof MENU_ITEMS)[number];
/*
type MenuItem = {
    label: string;
    path: string;
}
*/

// Med 'as const'
const MENU_ITEMS_CONST = [
  { label: "Hjem", path: "/" },
```

Const assertion

```
}
*/

// Med 'as const'
const MENU_ITEMS_CONST = [
  { label: "Hjem", path: "/" },
  { label: "Profil", path: "/profil" },
  { label: "Innstillinger", path: "/innstillinger" },
] as const;

type MenuItemConst = (typeof MENU_ITEMS_CONST)[number];
/*
type MenuItem = {
    readonly label: "Hjem";
    readonly path: "/";
} | {
    readonly label: "Profil";
    readonly path: "/profil";
} | {
    readonly label: "Innstillinger";
    readonly path: "/innstillinger";
}*/
```

Satisfies operator

```ts
const routes: Record<string, `/${string}`> = {
  hjem: "/",
  profil: "/users/:id",
  dashboard: "/dashboard",
};
/*
Record<string, `/${string}`>
Ulempe: typen blir bred, og vi mister informasjon om at det kun er strenger som starter med "/" 😕
*/

const routesWithoutTypes = {
  hjem: "/",
  profil: "/users/:id",
  dashboard: "/dashboard",
} as const;

/*
{
    hjem: "/";
    profil: "/users/:id";
    dashboard: "/dashboard";
}
Bedre, men ikke optimalt 🤔 Vi har mistet kravet om at routes må starte med "/"
```

Satisfies operator

```
Record<string, `/${string}`>
Ulempe: typen blir bred, og vi mister informasjon om at det kun er strenger som starter med "/" 😕
*/

const routesWithoutTypes = {
  hjem: "/",
  profil: "/users/:id",
  dashboard: "/dashboard",
} as const;

/*
{
    hjem: "/";
    profil: "/users/:id";
    dashboard: "/dashboard";
}
Bedre, men ikke optimalt 🤔 Vi har mistet kravet om at routes må starte med "/"
*/

const routesSatisfies = {
  hjem: "/",
  profil: "/users/:id",
  dashboard: "/dashboard",
```

Satisfies operator

```
/*
{
    hjem: "/";
    profil: "/users/:id";
    dashboard: "/dashboard";
}
Bedre, men ikke optimalt 🤔 Vi har mistet kravet om at routes må starte med "/"
*/

const routesSatisfies = {
  hjem: "/",
  profil: "/users/:id",
  dashboard: "/dashboard",
} satisfies Record<string, `/${string}`>;

/*
{
    hjem: "/";
    profil: "/users/:id";
    dashboard: "/dashboard";
}
Med satisfies har vi bevart type-sikkerhet og informasjon om at det kun er strenger som starter med "/" 🤩
*/
```

Bruk 'never' for å sørge for at alle valg er dekket i en switch-case

```ts
type Color = "red" | "green" | "blue" | "yellow";

const getColorHex = (color: Color): string => {
  switch (color) {
    case "red":
      return "#FF0000";
    case "green":
      return "#00FF00";
    case "blue":
      return "#0000FF";
    default:
      const _exhaustiveCheck: never = color;
      // 🚫 Type '"yellow"' is not assignable to type 'never'.ts(2322)
      return _exhaustiveCheck;
  }
};
```

'Never' for å filtrere ut uønskede typer

```ts
// Felles base
type BaseColumn = {
  id: string;
  label: string;
};

// Når kolonnen er redigerbar
type EditableColumn = BaseColumn & {
  isEditable: true;
  placeholder: string;
  onEdit: (value: string) ⇒ void;
};

// Når kolonnen IKKE er redigerbar
type NonEditableColumn = BaseColumn & {
  isEditable?: false | undefined;
  placeholder?: never;
  onEdit?: never;
};

// Kombinert variant
type Column = EditableColumn | NonEditableColumn;

```

'Never' for å filtrere ut uønskede typer

```ts
// Felles base
type BaseColumn = {
  id: string;
  label: string;
};

// Når kolonnen er redigerbar
type EditableColumn = BaseColumn & {
  isEditable: true;
  placeholder: string;
  onEdit: (value: string) ⇒ void;
};

// Når kolonnen IKKE er redigerbar
type NonEditableColumn = BaseColumn & {
  isEditable?: false | undefined;
  placeholder?: never;
  onEdit?: never;
};

// Kombinert variant
type Column = EditableColumn | NonEditableColumn;

```

'Never' for å filtrere ut uønskede typer

```typescript
// Felles base
type BaseColumn = {
  id: string;
  label: string;
};

// Når kolonnen er redigerbar
type EditableColumn = BaseColumn & {
  isEditable: true;
  placeholder: string;
  onEdit: (value: string) => void;
};

// Når kolonnen IKKE er redigerbar
type NonEditableColumn = BaseColumn & {
  isEditable?: false | undefined;
  placeholder?: never;
  onEdit?: never;
};

// Kombinert variant
type Column = EditableColumn | NonEditableColumn;

```

'Never' for å filtrere ut uønskede typer

```
    isEditable: true;
    placeholder: string;
    onEdit: (value: string) => void;
  };

  // Når kolonnen IKKE er redigerbar
  type NonEditableColumn = BaseColumn & {
    isEditable?: false | undefined;
    placeholder?: never;
    onEdit?: never;
  };

  // Kombinert variant
  type Column = EditableColumn | NonEditableColumn;

  const nameCol: Column = {
    id: "name",
    label: "Navn",
    isEditable: true,
    placeholder: "Skriv inn navn",
    onEdit: (v) => console.log(v),
  };

  const idCol: Column = {
```

'Never' for å filtrere ut uønskede typer

```
22  type Column = EditableColumn | NonEditableColumn;
23
24  const nameCol: Column = {
25    id: "name",
26    label: "Navn",
27    isEditable: true,
28    placeholder: "Skriv inn navn",
29    onEdit: (v) ⇒ console.log(v),
30  };
31
32  const idCol: Column = {
33    id: "id",
34    label: "Bruker-ID",
35    isEditable: false,
36  };
37
38  const badCol: Column = {
39    id: "email",
40    label: "E-post",
41    isEditable: false,
42    placeholder: "Ikke lov her", // ❌ Type '"Ikke lov her"' is not assignable to type 'never'
43    onEdit: () ⇒ {}, // ❌ Type '() ⇒ void' is not assignable to type 'undefined'
44  };
```

'Never' for å filtrere ut uønskede typer

```
    onEdit: (v) => console.log(v),
  };

  const idCol: Column = {
    id: "id",
    label: "Bruker-ID",
    isEditable: false,
  };

  const badCol: Column = {
    id: "email",
    label: "E-post",
    isEditable: false,
    placeholder: "Ikke lov her", // ❌ Type '"Ikke lov her"' is not assignable to type 'never'
    onEdit: () => {}, // ❌ Type '() => void' is not assignable to type 'undefined'
  };

  // Enda bedre versjon av NonEditableColumn 😍
  type BetterNonEditableColumn = BaseColumn & {
    isEditable: false;
  } & {
    [key in keyof Pick<EditableColumn, "placeholder" | "onEdit">]?: never;
  };
```

Infer for å ekstrahere typer

```ts
// Hent ut item type i en liste
const persons = [
  { name: "Ola Nordmann", age: 34 },
  { name: "Kari Hansen", age: 51 },
];

type ArrayItemType<T> = T extends Array<infer E> ? E : never;

type Person = ArrayItemType<typeof persons>;
/*
type Person = {
  name: string;
  age: number;
};
*/

// Fra React sin typedefinisjon
// @types/react/index.d.ts

type ContextType<C extends Context<any>> = C extends Context<infer T>
  ? T
  : never;
```

Infer for å ekstrahere typer

```ts
type Person = {
  name: string;
  age: number;
};
*/

// Fra React sin typedefinisjon
// @types/react/index.d.ts

type ContextType<C extends Context<any>> = C extends Context<infer T>
  ? T
  : never;

function createContext<T>(defaultValue: T): Context<T>;

const MyContext = createContext({ foo: "bar" });

type MyContextType = ContextType<typeof MyContext>;

/*
type MyContextType = { foo: string }
*/
```

Lagre en type i variabel for gjenbruk

```
1  import { CSSProperties } from "react";
2
3  type BaseTheme = {
4    color: {
5      primary: string;
6      secondary: string;
7      accent: string;
8      background: string;
9    };
10   font: {
11     family: string;
12     size: number;
13   };
14   spacing: {
15     xs: number;
16     sm: number;
17     md: number;
18     lg: number;
19     xl: number;
20   };
21 };
22
23 const createThemedStyle = <TTheme extends BaseTheme>() => {
```

Lagre en type i variabel for gjenbruk

```
    sm: number;
    md: number;
    lg: number;
    xl: number;
  };
};

const createThemedStyle = <TTheme extends BaseTheme>() => {
  const useStyle = (fn: (theme: TTheme) => CSSProperties) => {
    // Mock implementation
    return {} as CSSProperties;
  };

  return { useStyle };
};

// Lagre Theme i variabelen useStyle
const { useStyle } = createThemedStyle<BaseTheme>();

const containerStyle = useStyle((theme) => ({
  backgroundColor: theme.color.background,
  padding: theme.spacing.lg,
}));
```

Lagre en type i variabel for gjenbruk

```
23  const createThemedStyle = <TTheme extends BaseTheme>() => {
24    const useStyle = (fn: (theme: TTheme) => CSSProperties) => {
25      // Mock implementation
26      return {} as CSSProperties;
27    };
28
29    return { useStyle };
30  };
31
32  // Lagre Theme i variabelen useStyle
33  const { useStyle } = createThemedStyle<BaseTheme>();
34
35  const containerStyle = useStyle((theme) => ({
36    backgroundColor: theme.color.background,
37    padding: theme.spacing.lg,
38  }));
39
40  const buttonStyle = useStyle((theme) => ({
41    color: theme.color.primary,
42    padding: theme.spacing.md,
43    fontSize: theme.font.size,
44  }));
45
```

Lagre en type i variabel for gjenbruk

```
40  const buttonStyle = useStyle((theme) => ({
41    color: theme.color.primary,
42    padding: theme.spacing.md,
43    fontSize: theme.font.size,
44  }));
45
46  type CustomTheme = BaseTheme & {
47    custom: {
48      color: string;
49    };
50  };
51
52  // Her lagrer vi customTheme i variabelen useCustomStyle
53  const { useStyle: useCustomStyle } = createThemedStyle<CustomTheme>();
54
55  const customContainerStyle = useCustomStyle((theme) => ({
56    backgroundColor: theme.custom.color,
57  }));
58
59  const customButtonStyle = useCustomStyle((theme) => ({
60    color: theme.custom.color,
61    padding: theme.spacing.lg,
62  }));
```

Utility types

```ts
// Mange utility typer
type A = Awaited<Promise<string>>;
type PartialDemo = Partial<Todo>;
const obj2: Required<Props> = { a: 5 };
const todo: Readonly<Todo> = { ... };
type TodoPreview = Pick<Todo, "title" | "completed">;
type TodoPreview = Omit<Todo, "description">;
type T0 = Exclude<"a" | "b" | "c", "a">;
type T0 = Extract<"a" | "b" | "c", "a" | "f">;
type T0 = NonNullable<string | number | undefined>;
type T1 = Parameters<(s: string) => void>;
type T1 = ReturnType<(s: string) => string>;

// ... og enda flere


// Eksempel på bruk av 'Parameters' - utility typen
import { useVirtualizer, VirtualItem } from "@tanstack/vue-virtual";
import { computed, ref } from "vue";

interface Row {
  id: string;
  name: string;
```

Utility types

```
17  // Eksempel på bruk av 'Parameters' - utility typen
18  import { useVirtualizer, VirtualItem } from "@tanstack/vue-virtual";
19  import { computed, ref } from "vue";
20
21  interface Row {
22    id: string;
23    name: string;
24    // Indekserer attributtet "key" fra VirtualItem
25    key: VirtualItem["key"];
26  }
27
28  const rows = ref<Row[]>([]);
29  const virtualizedContainerRef = ref<HTMLElement | null>(null);
30
31  const rowVirtualizerOptions = computed(() => {
32    // Bruker utility-typen Parameters og henter ut første parameter
33    const options: Parameters<typeof useVirtualizer>[0] = {
34      count: rows.value.length,
35      getScrollElement: () => virtualizedContainerRef.value,
36      estimateSize: () => 28,
37      overscan: 50,
38      scrollPaddingStart: 150,
39      scrollPaddingEnd: 300,
```

Utility types

```
23      name: string;
24      // Indekserer attributtet "key" fra VirtualItem
25      key: VirtualItem["key"];
26    }
27
28    const rows = ref<Row[]>([]);
29    const virtualizedContainerRef = ref<HTMLElement | null>(null);
30
31    const rowVirtualizerOptions = computed(() => {
32      // Bruker utility-typen Parameters og henter ut første parameter
33      const options: Parameters<typeof useVirtualizer>[0] = {
34        count: rows.value.length,
35        getScrollElement: () => virtualizedContainerRef.value,
36        estimateSize: () => 28,
37        overscan: 50,
38        scrollPaddingStart: 150,
39        scrollPaddingEnd: 300,
40      };
41
42      return options;
43    });
44
45    const rowVirtualizer = useVirtualizer(rowVirtualizerOptions);
```

Discriminated Union

```
interface BaseTransaksjon {
  belop: number;
  dato: Date;
}

interface UttaksTransaksjon extends BaseTransaksjon {
  type: "uttak";
  fraKonto: string;
}

interface InnskuddsTransaksjon extends BaseTransaksjon {
  type: "innskudd";
  tilKonto: string;
}

interface OverføringsTransaksjon extends BaseTransaksjon {
  type: "overføring";
  fraKonto: string;
  tilKonto: string;
}

type Transaksjon =
  | UttaksTransaksjon
```

Discriminated Union

```
interface BaseTransaksjon {
  belop: number;
  dato: Date;
}

interface UttaksTransaksjon extends BaseTransaksjon {
  type: "uttak";
  fraKonto: string;
}

interface InnskuddsTransaksjon extends BaseTransaksjon {
  type: "innskudd";
  tilKonto: string;
}

interface OverføringsTransaksjon extends BaseTransaksjon {
  type: "overføring";
  fraKonto: string;
  tilKonto: string;
}

type Transaksjon =
  | UttaksTransaksjon
```

Discriminated Union

```
5
6   interface UttaksTransaksjon extends BaseTransaksjon {
7     type: "uttak";
8     fraKonto: string;
9   }
10
11  interface InnskuddsTransaksjon extends BaseTransaksjon {
12    type: "innskudd";
13    tilKonto: string;
14  }
15
16  interface OverføringsTransaksjon extends BaseTransaksjon {
17    type: "overføring";
18    fraKonto: string;
19    tilKonto: string;
20  }
21
22  type Transaksjon =
23    | UttaksTransaksjon
24    | InnskuddsTransaksjon
25    | OverføringsTransaksjon;
26
27  // Samme eksempel, men med Zod
```

Discriminated Union

```
// Samme eksempel, men med Zod
import { z } from "zod";

const baseTransaksjonSchema = z.object({
  belop: z.number(),
  dato: z.date(),
});

const uttaksTransaksjonSchema = baseTransaksjonSchema.extend({
  type: z.literal("uttak"),
  fraKonto: z.string(),
});

const innskuddsTransaksjonSchema = baseTransaksjonSchema.extend({
  type: z.literal("innskudd"),
  tilKonto: z.string(),
});

const overføringsTransaksjonSchema = baseTransaksjonSchema.extend({
  type: z.literal("overføring"),
  fraKonto: z.string(),
  tilKonto: z.string(),
});
```

Discriminated Union

```
35  const uttaksTransaksjonSchema = baseTransaksjonSchema.extend({
36    type: z.literal("uttak"),
37    fraKonto: z.string(),
38  });
39
40  const innskuddsTransaksjonSchema = baseTransaksjonSchema.extend({
41    type: z.literal("innskudd"),
42    tilKonto: z.string(),
43  });
44
45  const overføringsTransaksjonSchema = baseTransaksjonSchema.extend({
46    type: z.literal("overføring"),
47    fraKonto: z.string(),
48    tilKonto: z.string(),
49  });
50
51  const transaksjonSchema = z.discriminatedUnion("type", [
52    uttaksTransaksjonSchema,
53    innskuddsTransaksjonSchema,
54    overføringsTransaksjonSchema,
55  ]);
56
57  type Transaksjon = z.infer<typeof transaksjonSchema>;
```

Eksempler på Discriminated Union i Tanstack Query

```ts
// Eksempler på Discriminated union i Tanstack Query
// @tanstack/query-core/build/modern/hydration-[id].d.ts

interface QueryObserverPendingResult<TData = unknown, TError = DefaultError>
  extends QueryObserverBaseResult<TData, TError> {
  data: undefined;
  error: null;
  isError: false;
  isPending: true;
  isLoadingError: false;
  isRefetchError: false;
  isSuccess: false;
  isPlaceholderData: false;
  status: "pending";
}
interface QueryObserverLoadingResult<TData = unknown, TError = DefaultError>
  extends QueryObserverBaseResult<TData, TError> {
  data: undefined;
  error: null;
  isError: false;
  isPending: true;
  isLoading: true;
  isLoadingError: false;
  isRefetchError: false;
```

Eksempler på Discriminated Union i Tanstack Query

```
interface QueryObserverPendingResult<TData = unknown, TError = DefaultError>
  extends QueryObserverBaseResult<TData, TError> {
  data: undefined;
  error: null;
  isError: false;
  isPending: true;
  isLoadingError: false;
  isRefetchError: false;
  isSuccess: false;
  isPlaceholderData: false;
  status: "pending";
}
interface QueryObserverLoadingResult<TData = unknown, TError = DefaultError>
  extends QueryObserverBaseResult<TData, TError> {
  data: undefined;
  error: null;
  isError: false;
  isPending: true;
  isLoading: true;
  isLoadingError: false;
  isRefetchError: false;
  isSuccess: false;
  isPlaceholderData: false;
  status: "pending";
}
```

Eksempler på Discriminated Union i Tanstack Query

```
    isRefetchError: false;
    isSuccess: false;
    isPlaceholderData: false;
    status: "pending";
  }
  interface QueryObserverLoadingErrorResult<
    TData = unknown,
    TError = DefaultError
  > extends QueryObserverBaseResult<TData, TError> {
    data: undefined;
    error: TError;
    isError: true;
    isPending: false;
    isLoading: false;
    isLoadingError: true;
    isRefetchError: false;
    isSuccess: false;
    isPlaceholderData: false;
    status: "error";
  }
  interface QueryObserverRefetchErrorResult<
    TData = unknown,
    TError = DefaultError
  > extends QueryObserverBaseResult<TData, TError> {
    data: TData;
```

Eksempler på Discriminated Union i Tanstack Query

```
    isRefetchError: false;
    isSuccess: false;
    isPlaceholderData: false;
    status: "error";
  }
  interface QueryObserverRefetchErrorResult<
    TData = unknown,
    TError = DefaultError
  > extends QueryObserverBaseResult<TData, TError> {
    data: TData;
    error: TError;
    isError: true;
    isPending: false;
    isLoading: false;
    isLoadingError: false;
    isRefetchError: true;
    isSuccess: false;
    isPlaceholderData: false;
    status: "error";
  }
  interface QueryObserverSuccessResult<TData = unknown, TError = DefaultError>
    extends QueryObserverBaseResult<TData, TError> {
    data: TData;
    error: null;
    isError: false;
```

Eksempler på Discriminated Union i Tanstack Query

```
    isLoadingError: false;
    isRefetchError: true;
    isSuccess: false;
    isPlaceholderData: false;
    status: "error";
  }
  interface QueryObserverSuccessResult<TData = unknown, TError = DefaultError>
    extends QueryObserverBaseResult<TData, TError> {
    data: TData;
    error: null;
    isError: false;
    isPending: false;
    isLoading: false;
    isLoadingError: false;
    isRefetchError: false;
    isSuccess: true;
    isPlaceholderData: false;
    status: "success";
  }
  interface QueryObserverPlaceholderResult<TData = unknown, TError = DefaultError>
    extends QueryObserverBaseResult<TData, TError> {
    data: TData;
    isError: false;
    error: null;
    isPending: false;
```

Eksempler på Discriminated Union i Tanstack Query

```
66      isLoadingError: false;
67      isRefetchError: false;
68      isSuccess: true;
69      isPlaceholderData: false;
70      status: "success";
71    }
72    interface QueryObserverPlaceholderResult<TData = unknown, TError = DefaultError>
73      extends QueryObserverBaseResult<TData, TError> {
74      data: TData;
75      isError: false;
76      error: null;
77      isPending: false;
78      isLoading: false;
79      isLoadingError: false;
80      isRefetchError: false;
81      isSuccess: true;
82      isPlaceholderData: true;
83      status: "success";
84    }
85
86    type QueryObserverResult<TData = unknown, TError = DefaultError> =
87      | DefinedQueryObserverResult<TData, TError>
88      | QueryObserverLoadingErrorResult<TData, TError>
89      | QueryObserverLoadingResult<TData, TError>
90      | QueryObserverPendingResult<TData, TError>
```

Eksempler på Discriminated Union i Tanstack Query

```
77      isPending: false;
78      isLoading: false;
79      isLoadingError: false;
80      isRefetchError: false;
81      isSuccess: true;
82      isPlaceholderData: true;
83      status: "success";
84    }
85
86    type QueryObserverResult<TData = unknown, TError = DefaultError> =
87      | DefinedQueryObserverResult<TData, TError>
88      | QueryObserverLoadingErrorResult<TData, TError>
89      | QueryObserverLoadingResult<TData, TError>
90      | QueryObserverPendingResult<TData, TError>
91      | QueryObserverPlaceholderResult<TData, TError>;
92
93    // Bruker discriminated union for å ugyldiggjøre branches uten å ha getNext..
94    // Tanstack Query @tanstack/query-core/build/modern/hydration-[id].d.ts
95    type FetchInfiniteQueryPages<TQueryFnData = unknown, TPageParam = unknown> =
96      | {
97          pages?: never;
98        }
99      | {
100         pages: number;
101         getNextPageParam: GetNextPageParamFunction<TPageParam, TQueryFnData>;
```

Eksempler på Discriminated Union i Tanstack Query

```
    isLoadingError: false;
    isRefetchError: false;
    isSuccess: true;
    isPlaceholderData: true;
    status: "success";
  }

  type QueryObserverResult<TData = unknown, TError = DefaultError> =
    | DefinedQueryObserverResult<TData, TError>
    | QueryObserverLoadingErrorResult<TData, TError>
    | QueryObserverLoadingResult<TData, TError>
    | QueryObserverPendingResult<TData, TError>
    | QueryObserverPlaceholderResult<TData, TError>;

  // Bruker discriminated union for å ugyldiggjøre branches uten å ha getNext..
  // Tanstack Query @tanstack/query-core/build/modern/hydration-[id].d.ts
  type FetchInfiniteQueryPages<TQueryFnData = unknown, TPageParam = unknown> =
    | {
        pages?: never;
      }
    | {
        pages: number;
        getNextPageParam: GetNextPageParamFunction<TPageParam, TQueryFnData>;
      };
```

Mapped lookup type

```ts
type FeltTypeMap = {
  tall: number;
  tekst: string;
  prosent: number;
  flervalg: {
    id: number;
    beskrivelse: string;
  };
};

type FeltKeys = keyof FeltTypeMap;
// type FeltKeys = "tall" | "tekst" | "prosent" | "flervalg"

type Felt<T extends FeltKeys> = {
  verdi: FeltTypeMap[T];
  valider: (verdi: FeltTypeMap[T]) => boolean;
  formatterVisning: (verdi: FeltTypeMap[T]) => string;
};

const opprettFelt = <T extends FeltKeys = never>(params: Felt<T>) => {
  return {
    verdi: params.verdi,
    valider: params.valider,
```

Mapped lookup type

```ts
type FeltTypeMap = {
  tall: number;
  tekst: string;
  prosent: number;
  flervalg: {
    id: number;
    beskrivelse: string;
  };
};

type FeltKeys = keyof FeltTypeMap;
// type FeltKeys = "tall" | "tekst" | "prosent" | "flervalg"

type Felt<T extends FeltKeys> = {
  verdi: FeltTypeMap[T];
  valider: (verdi: FeltTypeMap[T]) => boolean;
  formatterVisning: (verdi: FeltTypeMap[T]) => string;
};

const opprettFelt = <T extends FeltKeys = never>(params: Felt<T>) => {
  return {
    verdi: params.verdi,
    valider: params.valider,
```

Mapped lookup type

```ts
type FeltTypeMap = {
  tall: number;
  tekst: string;
  prosent: number;
  flervalg: {
    id: number;
    beskrivelse: string;
  };
};

type FeltKeys = keyof FeltTypeMap;
// type FeltKeys = "tall" | "tekst" | "prosent" | "flervalg"

type Felt<T extends FeltKeys> = {
  verdi: FeltTypeMap[T];
  valider: (verdi: FeltTypeMap[T]) => boolean;
  formatterVisning: (verdi: FeltTypeMap[T]) => string;
};

const opprettFelt = <T extends FeltKeys = never>(params: Felt<T>) => {
  return {
    verdi: params.verdi,
    valider: params.valider,
```

Mapped lookup type

```
  };
};

type FeltKeys = keyof FeltTypeMap;
// type FeltKeys = "tall" | "tekst" | "prosent" | "flervalg"

type Felt<T extends FeltKeys> = {
  verdi: FeltTypeMap[T];
  valider: (verdi: FeltTypeMap[T]) => boolean;
  formatterVisning: (verdi: FeltTypeMap[T]) => string;
};

const opprettFelt = <T extends FeltKeys = never>(params: Felt<T>) => {
  return {
    verdi: params.verdi,
    valider: params.valider,
    formatterVisning: params.formatterVisning,
  };
};

const prosentFelt = opprettFelt<"prosent">({
  verdi: 0.25,
  valider: (x) => x >= -1 && x <= 1,
```

Mapped lookup type

```
20  const opprettFelt = <T extends FeltKeys = never>(params: Felt<T>) => {
21    return {
22      verdi: params.verdi,
23      valider: params.valider,
24      formatterVisning: params.formatterVisning,
25    };
26  };
27
28  const prosentFelt = opprettFelt<"prosent">({
29    verdi: 0.25,
30    valider: (x) => x >= -1 && x <= 1,
31    formatterVisning: (x) => x.toString(),
32  });
33
34  const tekstFelt = opprettFelt<"tekst">({
35    verdi: "hei",
36    valider: (x) => x.length > 0,
37    formatterVisning: (x) => x,
38  });
39
40  const tallFelt = opprettFelt<"tall">({
41    verdi: 123,
42    valider: (x) => x > 0,
```

Mapped lookup type

```
28  const prosentFelt = opprettFelt<"prosent">({
29    verdi: 0.25,
30    valider: (x) ⇒ x ⩾ -1 && x ⩽ 1,
31    formatterVisning: (x) ⇒ x.toString(),
32  });
33
34  const tekstFelt = opprettFelt<"tekst">({
35    verdi: "hei",
36    valider: (x) ⇒ x.length > 0,
37    formatterVisning: (x) ⇒ x,
38  });
39
40  const tallFelt = opprettFelt<"tall">({
41    verdi: 123,
42    valider: (x) ⇒ x > 0,
43    formatterVisning: (x) ⇒ x.toString(),
44  });
45
46  const flervalgFelt = opprettFelt<"flervalg">({
47    verdi: { beskrivelse: "hei", id: 5 },
48    valider: (x) ⇒ x.beskrivelse.length > 0,
49    formatterVisning: (x) ⇒ `${x.id} - ${x.beskrivelse}`,
50  });
```

Vi lager en kolonnefabrikk 🏭

```typescript
interface OptionValue {
  value: string | number;
  label: string;
}

type ValueTypeMap = {
  text: string | undefined;
  number: number | undefined | null;
  date: string | Date | null;
  option: OptionValue | undefined;
};

type ColumnType = keyof ValueTypeMap;

type BaseColumnProps<TData, TType extends ColumnType> = {
  id: keyof TData;
  header: string;
  accessorFn: (row: TData) => ValueTypeMap[TType];
  sortable?: boolean;
  filterable?: boolean;
  size?: number;
  render?: (value: ValueTypeMap[TType], row: TData) => string;
};
```

Module Augmentation

```ts
// Preliminaries - merging av interfaces
interface Person {
  navn: string;
  alder: number;
}

interface Person {
  gate: string;
  postnummer: number;
  poststed: string;
}

interface Person {
  telefonnummer: string;
}

const person: Person = {
  navn: "Ola Nordmann",
  alder: 40,
  gate: "Osloveien 12",
  postnummer: 7018,
  poststed: "Trondheim",
  telefonnummer: "98765432",
};
```

Tusen takk takk for at dere kom, også håper jeg at noen av dere har fått utvidet komfortsonen litt når det kommer til TypeScript 🙌🏻